Basic image compression methods, introduction to JPEG and JPEG 2000 standards

Karel Fliegel
(fliegek@fel.cvut.cz)

MultiMedia Technology Group

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Radioelectronics

Technická 2
166 27 Prague 6
Czech Republic

24/4/2026

© Multimedia Technology Group

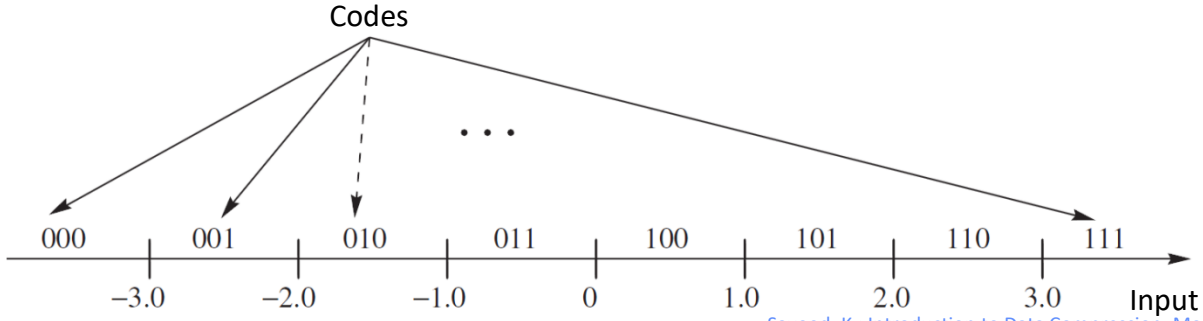
Contents

- **Basic lossy image compression methods**
  - Scalar quantization
  - Vector quantization
  - Differential encoding
- **Transform coding**
  - Kahunen-Loéve transform (KLT),
    discrete cosine transform (DCT), discrete sine transform (DST),
    discrete Walsh-Hadamard transform (WHT)
  - Subband coding
  - Wavelet transform
- **Introduction to image compression standards**
  - Standard JPEG
  - Standard JPEG 2000

Basic lossy image compression methods: Scalar quantization

Scalar quantization

- One of **simplest „lossy compression techniques“**
- Mapping **large set** (possibly infinite) of numbers to much **smaller set**
  - **Inaccuracies** appear – kind of **lossy compression** scheme
- **Quantization** process is very **simple**
  - **Crucial impact** on **amount** of **loss**
- **Part** of **A/D conversion**
- **Example**
  - Mapping **codewords** for **3-bit quantization** (encoding)

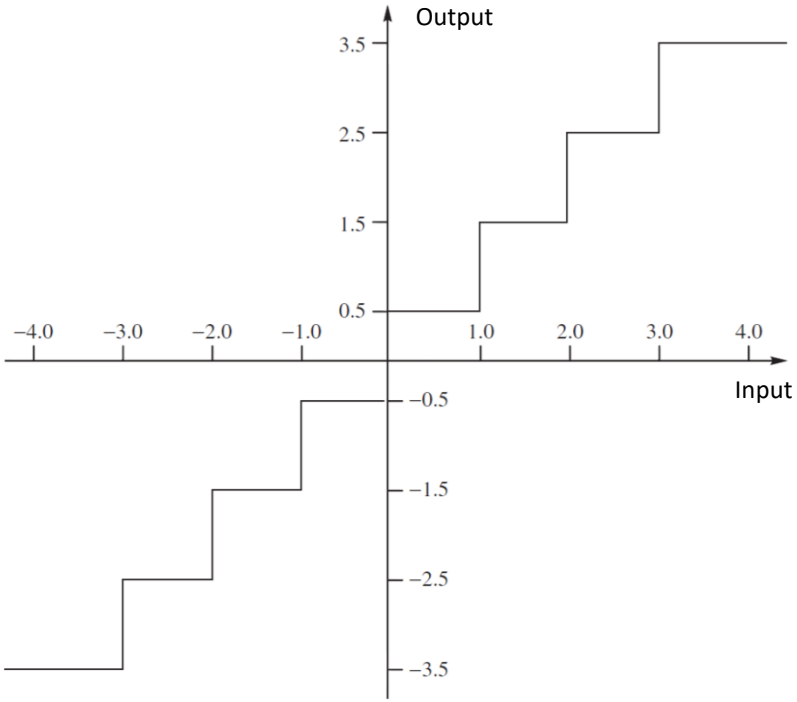


Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Scalar quantization

Example

- Mapping **codewords** for **3-bit quantization** (encoding)
  - Selection of **intervals** and **output values** is part of **quantizer design**
  - Selection has **impact** on **fidelity** of reconstructed signal



Quantizer input-output map

Mapping for 3-bit D/A converter

Input codes	Output
000	−3.5
001	−2.5
010	−1.5
011	−0.5
100	0.5
101	1.5
110	2.5
111	3.5

Digitizing sine wave

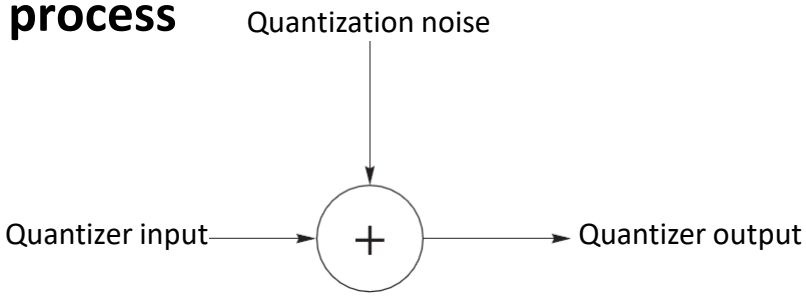
t	$4\cos(2\pi t)$	A/D output	D/A output	Error
0.05	3.804	111	3.5	0.304
0.10	3.236	111	3.5	−0.264
0.15	2.351	110	2.5	−0.149
0.20	1.236	101	1.5	−0.264

© Multimedia Technology Group

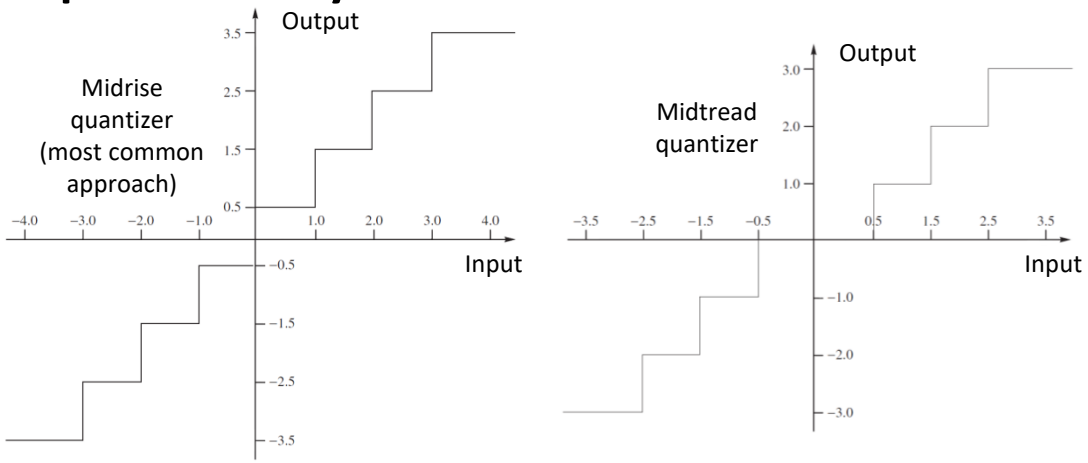
Basic lossy image compression methods: Scalar quantization

Quantization noise

- Quantization **causes quantization (error) noise**
  - Given as **difference** between **quantizer input** x and **output** $y = Q(x)$
  - Quantization is **modeled** as **additive process**



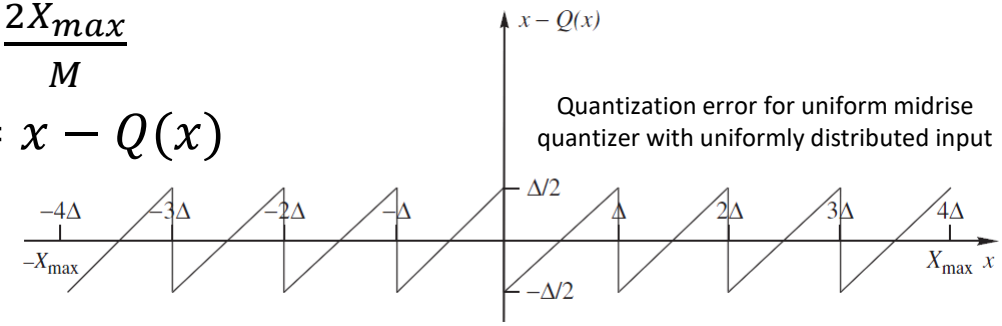
- **Uniform quantizer** as simplest example
  - **Decision boundaries** are **spaced evenly**
  - **Quantization step** is **uniform**
  - Various **solutions** for quantization **characteristic**
    - Impact on **system properties**



Basic lossy image compression methods: Scalar quantization

Uniform quantization of uniformly distributed source

- Number of **quantization levels** $M = 2^n$, **input signal** $[-X_{max}, +X_{max}]$
- Quantization **step** $\Delta = \frac{2X_{max}}{M}$
- Quantization **error** $q = x - Q(x)$



- **Mean squared** value of **quantization error**

$$\sigma_q^2 = \frac{1}{\Delta}\int_{-\frac{\Delta}{2}}^{\frac{\Delta}{2}} q^2 dq = \frac{\Delta^2}{12}$$

- Signal **variance** for **uniform random variable** $\sigma_s^2 = \frac{(2X_{max})^2}{12}$
- **Signal** to **noise ratio**

$$SNR = 10\log_{10}\frac{\sigma_s^2}{\sigma_q^2} = 10\log_{10}\left(\frac{(2X_{max})^2}{12}\frac{12}{\Delta^2}\right) = 10\log_{10}M^2 = 20\log_{10}2^n$$

$$= 6.02\, n \text{ dB}$$

© Multimedia Technology Group

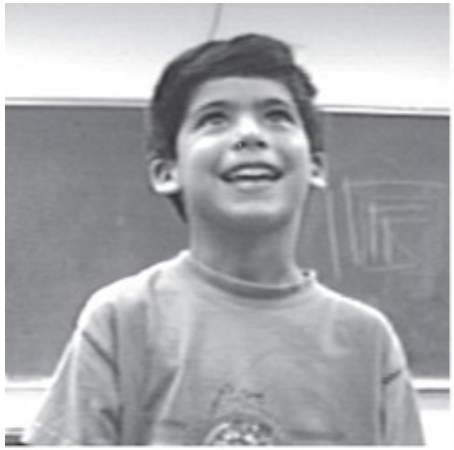
Image compression example

- **Common range** 0 to $2^b - 1$ with expected **uniform distribution** of values

Original 8 bpp image

Quantization 1 bpp

256 levels



2 levels
Boundary values e.g.
$\{0, 128, 255\}$
Reconstruction values e.g.
$\{64, 196\}$

4 levels
Boundary values e.g.
$\{0, 64, 128, 196, 255\}$
Reconstruction values e.g.
$\{32, 96, 160, 224\}$

Disturbing **contouring after quantization** can be reduced using **dithering**

8 levels

Quantization 2 bpp

Quantization 3 bpp

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Basic lossy image compression methods: Scalar quantization

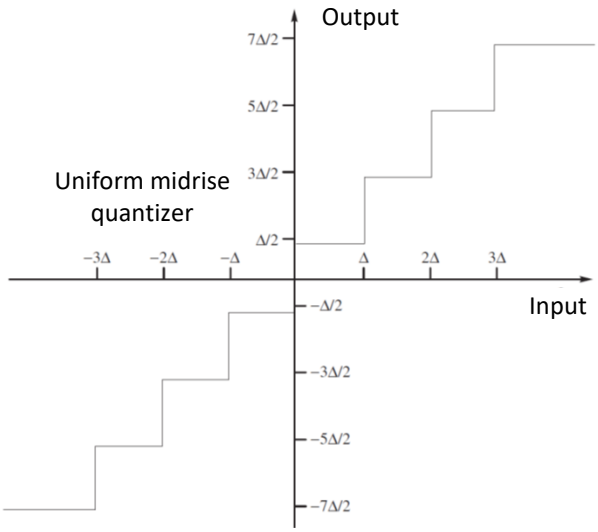
Uniform quantization of nonuniform sources

- Most sources with **nonuniform distribution of values**
  - Application of **uniform quantization** provides **unsatisfactory result**
  - **Quantizer design** should take into account **distribution** $f_x(x)$
  - **Quantization step** to **minimize quantization noise**

$$\sigma_q^2 = 2\sum_{i=1}^{\frac{M}{2}-1}\int_{(i-1)\Delta}^{i\Delta}\left(x-\frac{2i-1}{2}\Delta\right)^2 f_x(x)dx + 2\int_{\left(\frac{M}{2}-1\right)\Delta}^{\infty}\left(x-\frac{M-1}{2}\Delta\right)^2 f_x(x)dx$$

$$\frac{d\sigma_q^2}{d\Delta} = -\sum_{i=1}^{\frac{M}{2}-1}(2i-1)\int_{(i-1)\Delta}^{i\Delta}\left(x-\frac{2i-1}{2}\Delta\right)f_x(x)dx$$

$$-(M-1)\int_{\left(\frac{M}{2}-1\right)\Delta}^{\infty}\left(x-\frac{M-1}{2}\Delta\right)f_x(x)dx = 0$$



- Solving **optimal quantization step** Δ for **given distribution** using **numerical methods**

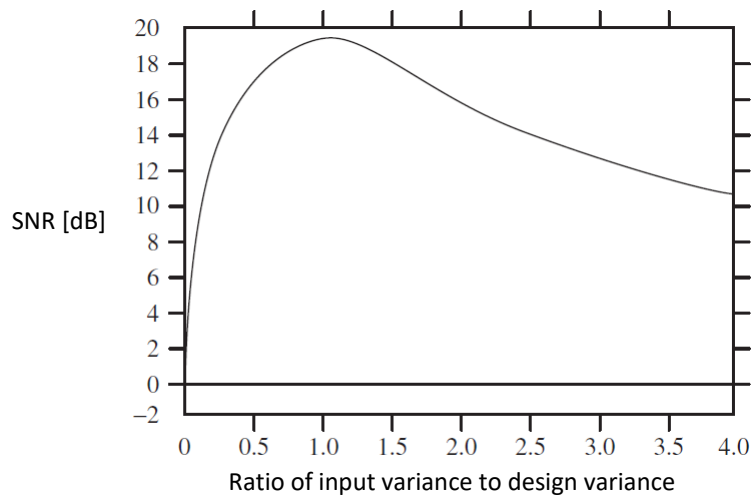
Basic lossy image compression methods: Scalar quantization

Uniform quantization of nonuniform sources

- Example of **optimal quantization** step and **SNR** for **various distributions**

Number of levels	Uniform distribution		Gaussian distribution		Laplacian distribution	
	Step Δ	SNR [dB]	Step Δ	SNR [dB]	Step Δ	SNR [dB]
2	1.732	6.02	1.596	4.40	1.414	3.00
4	0.866	12.04	0.9957	9.24	1.0873	7.05
6	0.577	15.58	0.7334	12.18	0.8707	9.56
8	0.433	18.06	0.5860	14.27	0.7309	11.39
10	0.346	20.02	0.4908	15.90	0.6334	12.81
12	0.289	21.60	0.4238	17.25	0.5613	13.98
14	0.247	22.94	0.3739	18.37	0.5055	14.98
16	0.217	24.08	0.3352	19.36	0.4609	15.84
32	0.108	30.10	0.1881	24.56	0.2799	20.46

- Effect of **worsened performance** for **variance mismatch**
  - Impact of **signal variance different** from **design assumption**
  - Example for **4-bit** Gaussian **uniform quantizer**



Basic lossy image compression methods: Scalar quantization

Adaptive quantization

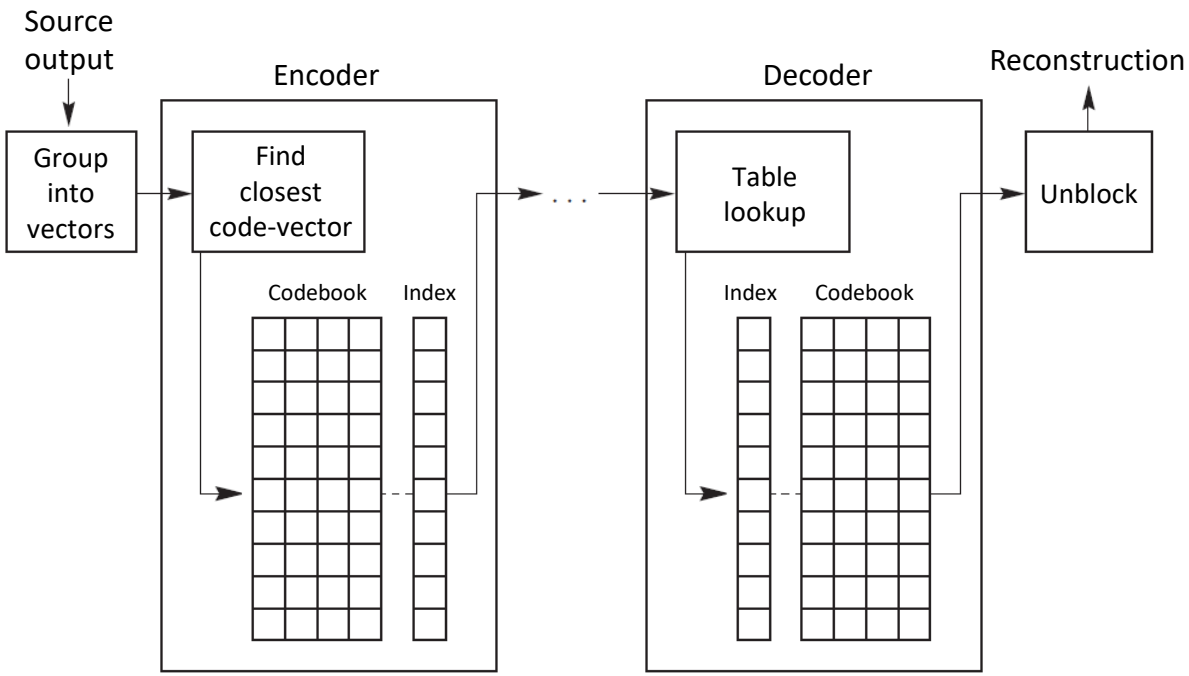
- Quantizer is **adapted** to input **signal characteristics**
  - **Matched** to **input signal statistics**, e.g. mean, variance, PDF
- **Two** main **approaches**
  - **Adaptive** quantizer with **forward adaptive** approach (*offline*)
    - Data blocks analyzed **before quantization**
    - **Optimization** of **quantization parameters** and **transmission** of **side data**
  - **Adaptive** quantizer with **backward adaptive** approach (*online*)
    - Quantizer **optimization** based on **quantized data**
    - Information **available** at **receiver side** – **no** need for **side data transmission**

Nonuniform quantization

- **Variable quantization step size** based on **signal distribution**
  - **Optimization** of **quantization steps** in respect to **signal PDF**
    - **Lloyd-Max** algorithm

Vector quantization

- **Efficient** lossy/lossles **compression** based on **quantization**
  - **Merging source output** into **blocks** and their encoding as **vectors**
  - **Quantization** of **sequences** is **more efficient** than **independent samples**
  - Every **sequence** is described by **element** from **characteristic set**



Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Basic lossy image compression methods: Vector quantization

Vector quantization terminology

- **Codebook** $\mathcal{C}$ with size K of code-vectors
- **Dimension** of input **vector** L
- **Number of bits** for information transmission of **selected vector** $\lceil \log_2 K \rceil$
  - For example **codebook** with $K = 256$ requires 8 **bits per vector**
- **Vector** carries **information** about L **source samples**
  - Then **number of bits per sample** $\lceil \log_2 K \rceil / L$

- **Codebook** $\mathcal{C}$ contains **code-vectors** $\{\boldsymbol{y}_i\}$,
  vector $\boldsymbol{x}$ **closest** to vector $\boldsymbol{y}_j$

$$\left\|\boldsymbol{x} - \boldsymbol{y}_j\right\|^2 \leq \|\boldsymbol{x} - \boldsymbol{y}_i\|^2 \text{ for all } \boldsymbol{y}_i \in \mathcal{C}$$

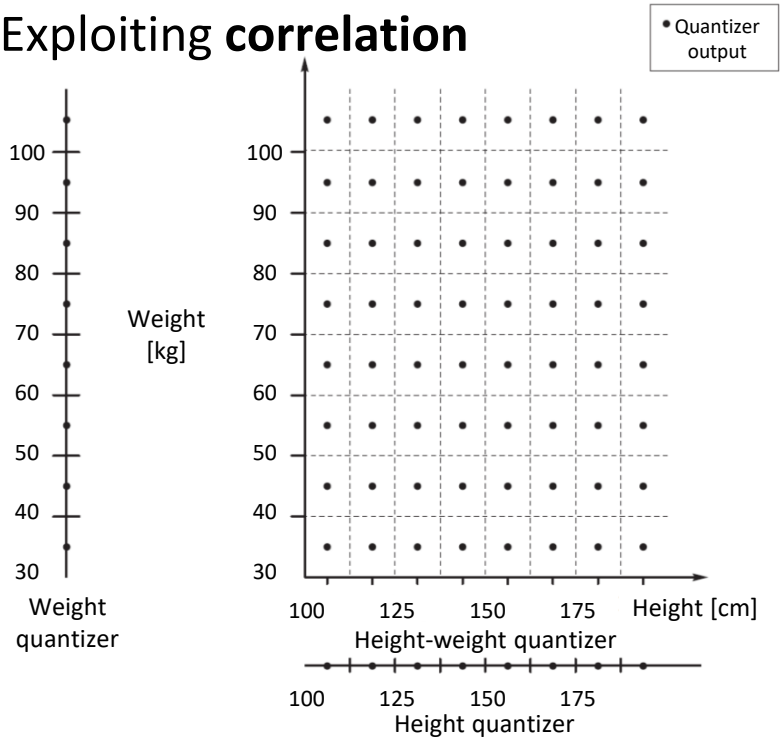
where $\boldsymbol{x} = (x_1, x_2, \dots, x_L)$ and

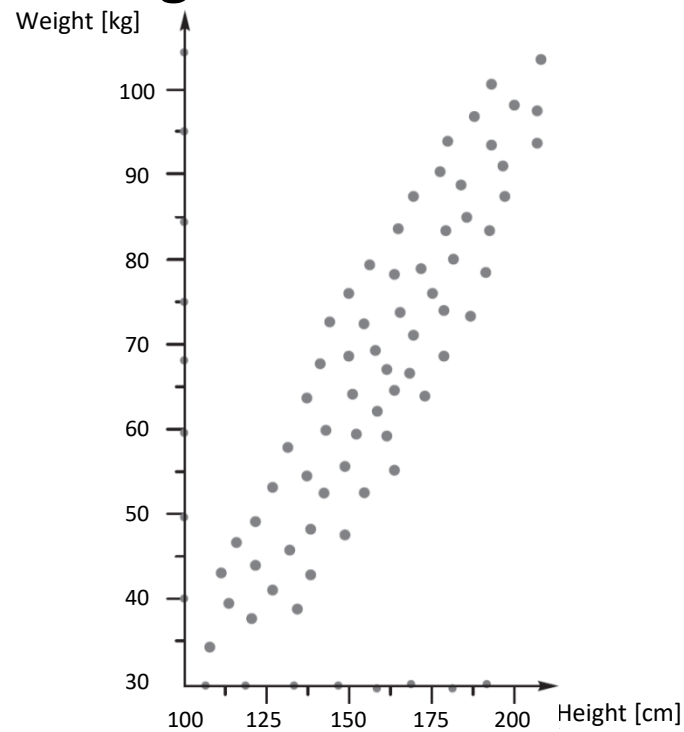
$$\|\boldsymbol{x}\|^2 = \sum_{i=1}^{L} x_i^2 .$$

- In **image compression** respective **samples** are represented as **pixels**

Basic lossy image compression methods: Vector quantization

- **Advantages of vector quantization over scalar quantization**
  - For the **same number of bits per sample** (pixel) **smaller distortion** than for **scalar quantization**
  - For **correlated source outputs**, **vector** representation creates **clusters** → codebook **vectors** within **clusters** → better representation
- Example **quantization** body **weight** and **height**
  - Exploiting **correlation**





Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Basic lossy image compression methods: Vector quantization

❖ **Linde-Buzo-Gray algorithm** (LBG algorithm)

- Automated **codebook design**
- **k-means clustering algorithm**, search for **clusters** and their **centroids**

1. Definition of **initial set** of reconstruction values (centroids) $\left\{\boldsymbol{y}_i^{(1)}\right\}_{i=1}^{M}$ and **training vectors** $\left\{\boldsymbol{x}_n\right\}_{n=1}^{N}$. Set $k = 1$, $D^{(0)} = 0$ and **select threshold** ε.

2. Find **quantization (Voronoi) regions** $\left\{V_i^{(1)}\right\}_{i=1}^{M}$ given as

$$V_i^{(k)} = \left\{\boldsymbol{x}_n : \boldsymbol{d}\left(\boldsymbol{x}_n, \boldsymbol{y}_i\right) < \boldsymbol{d}\left(\boldsymbol{x}_n, \boldsymbol{y}_j\right) \forall j \neq i\right\} \qquad i = 1,2,\dots,M$$

Assuming none of the regions is empty.

3. Compute distortion $D^{(k)}$ between **training vectors** and **centroids**

$$D^{(k)} = \frac{1}{N}\sum_{i=1}^{M}\sum_{\boldsymbol{x}_n \in V_i^{(k)}}\left\|\boldsymbol{x}_n - \boldsymbol{y}_j\right\|^2$$

4. Stop here if **does not converge fast** $\frac{\left(D^{(k)} - D^{(k-1)}\right)}{D^{(k)}} < \varepsilon$.

5. Set $k = k + 1$, find new **centroids** $\left\{\boldsymbol{y}_i^{(k)}\right\}_{i=1}^{M}$ as mean values in regions $V_i^{(k-1)}$.

Go to step 2.

Basic lossy image compression methods: Vector quantization

- **Linde-Buzo-Gray algorithm** (LBG algorithm)
  - Important **initial selection** of **centroids**
  - Can be solved using **LBG algorithm** with **sequential partitioning**

1. Design of vector quantizer for **codebook size** $M = 1$. Can be calculated as centroid (center of mass).

2. Partitioning **into two regions** using **small shift** of vector $\boldsymbol{\varepsilon}$.

3. Application of **LBG algorithm** for quantizer with codebook size $M = 2$.

4. Two obtained codebook vectors **partitioned into four** (similarly as in step 2) using **small shift** of vector $\boldsymbol{\varepsilon}$.

5. Application of **LBG algorithm** for quantizer with codebook size $M = 4$.

6. Two above steps repeated with sequential **partitioning** and reaching **required size** of codebook.

- Using **last codebook** in each step → **at least as good representation as** the previous codebook with **half size**

Basic lossy image compression methods: Vector quantization

❖ LBG algorithm application for image compression

- Input **image divided** into **blocks** of image pixels with size $M \times N$
- Each **block** is converted **into vector** with length $L = M \times N$
- Necessary **codebook transmission** to receiver

Division into blocks 4×4
Vectors with dimension 16

Codebook size 16
4 bits per codeword
0.25 bpp
Compression ratio 32:1

Division into blocks 4×4
Vectors with dimension 16

Codebook size 64
6 bits per codeword
0.375 bpp
Compression ratio 21.33:1

Division into blocks 4×4
Vectors with dimension 16

Codebook size 256
8 bits per codeword
0.50 bpp
Compression ratio 16:1

Division into blocks 4×4
Vectors with dimension 16

Codebook size 1024
10 bits per codeword
0.625 bpp
Compression ratio 12.8:1

© Multimedia Technology Group

Basic lossy image compression methods: Differential encoding

Differential encoding

- High **sample** to **sample correlation** for **image** signals
- **Prediction** using **previous values** and **difference transmission**
- **Simple implementation** and **low computational complexity**
- Example of **image properties** (dynamic range of differences)

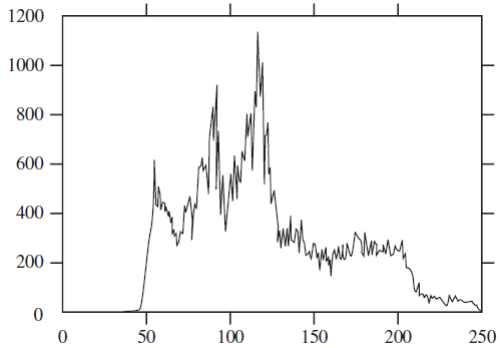
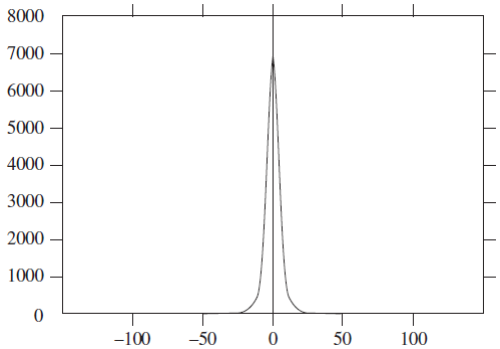


Image histogram



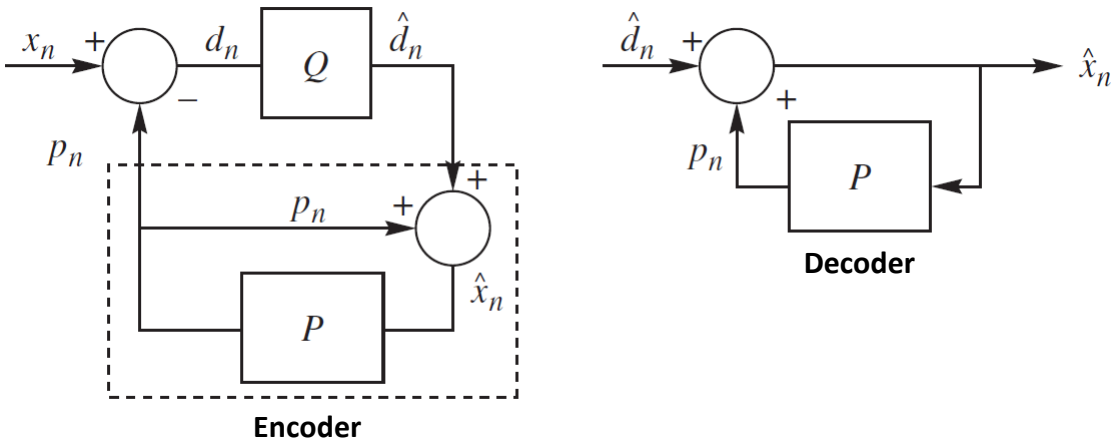
Histogram of pixel to pixel differences

- For **grayscale representation 8 bits per pixel** required
  - For **error on LSB level** enough **7 bits per pixel**
- Pixel to pixel differences **99% histogram values** in **range -31 to 31**
  - For **error on LSB level** for **99% pixels** enough **5 bits per pixel**

Basic lossy image compression methods: Differential encoding

DPCM scheme (*differencial pulse code modulation*)

Differential pulse code modulation



- **Simplest prediction** $p_n = f(\hat{x}_{n-1}, \hat{x}_{n-2}, \dots, \hat{x}_0)$ using **previously reconstructed** sample $p_n = \hat{x}_{n-1}$, then for **individual iterations**

$$d_n = x_n - \hat{x}_{n-1}$$

$$\hat{x}_0 = x_0, \qquad d_1 = x_1 - x_0, \qquad \hat{d}_1 = Q[d_1] = d_1 + q_1,$$

$$\hat{x}_1 = x_0 + \hat{d}_1 = x_0 + d_1 + q_1 = x_1 + q_1$$

$$d_2 = x_2 - \hat{x}_1, \qquad \hat{d}_2 = Q[d_2] = d_2 + q_2,$$

$$\hat{x}_2 = \hat{x}_1 + d_2 + q_2 = x_2 + q_2$$

- For **iteration** n applies $\hat{x}_n = x_n + q_n$

Basic lossy image compression methods: Differential encoding

DPCM application for direct image compression

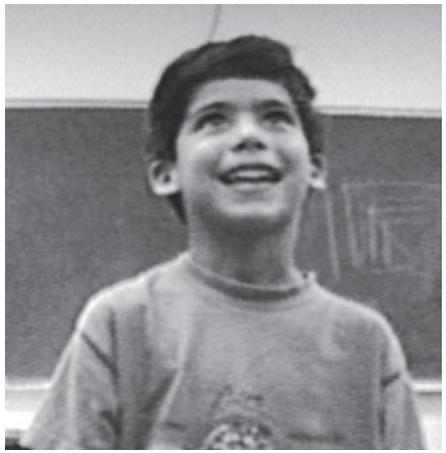
- **DPCM** included in **more complex compression schemes**
  - For example in JPEG for **decorrelation** of **low frequency components**

- Simple **pixel predictor** $p[j,k]$
  - Based on **previously reconstructed pixels** $\hat{x}[j,k]$ in row j and column k

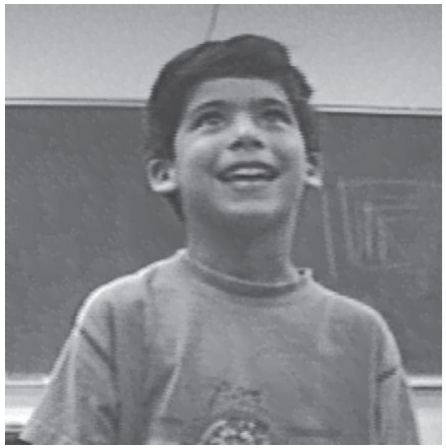
$$p[j,k] = \begin{cases} \hat{x}[j,k-1] & k > 0 \\ \hat{x}[j-1,k] & k = 0, j > 0 \\ 128 & j = 0, k = 0 \end{cases}$$

- More sophisticated **median predictor** $p[j,k]$

$$p_1 = 0.5\hat{x}[j-1,k] + 0.5\hat{x}[j,k-1]$$
$$p_2 = 0.5\hat{x}[j-1,k-1] + 0.5\hat{x}[j,k-1]$$
$$p_3 = 0.5\hat{x}[j-1,k-1] + 0.5\hat{x}[j-1,k]$$
$$p[j,k] = \text{median}\{p_1, p_2, p_3\}$$



1 bpp, PSNR = 31.42 dB



1 bpp, PSNR = 38.28 dB

Transform coding: Introduction

- **Transform coding**
  - Input signal **transformed** and then **encoded**
  - In **image compression** prominence of **DCT –** used in **JPEG**
    - Transformation allows for **compact representation** (decorrelation)
- **Transform** (two dimensional example)
  - Common **transforms** are **linear** and convert block $X_{i,j}$ into block $\Theta_{k,l}$ - **forward transform**

$$\Theta_{k,l} = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} X_{i,j} a_{i,j,k,l}$$

  - Common **transforms** are often **separable**

$$\Theta_{k,l} = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} a_{k,i} X_{i,j} a_{j,l}$$

    - Important characteristic is **variance** of individual **coefficients** σ_n^2
    - **Block size** $N \times N$ is determined mainly by **practical requirements**

Transform coding: Transforms

Transform in matrix representation

- **Forward** and **inverse transform** can be written in matrix form

$$\Theta = \mathbf{A}\mathbf{X}\mathbf{A}^T, [\mathbf{A}]_{k,i} = a_{k,i}$$
$$\mathbf{X} = \mathbf{B}\Theta\mathbf{B}^T, [\mathbf{B}]_{k,i} = b_{k,i}$$

where $\mathbf{A}$ and $\mathbf{B}$ are **transformation matrices** for **forward** and **inverse** transform of size $N \times N$.

- Transforms are **orthonormal** where applies

$$\mathbf{B} = \mathbf{A}^{-1} = \mathbf{A}^T$$

- For **orthonormal** transform its **inverse** variant is

$$\mathbf{X} = \mathbf{A}^T\Theta\mathbf{A}$$

- **Orthonormal** transforms **preserve energy**
- **Efficiency** of transform and its **conversion ability** to provide **compact representation**
  - Can be described as **ratio** between **arithmetic** and **geometric mean** of **coefficient** variances (transform coding gain)

$$G_{TC} = \frac{1}{N}\sum_{i=0}^{N-1}\sigma_i^2 \Big/ \prod_{i=0}^{N-1}(\sigma_i^2)^{\frac{1}{N}}$$

Transform coding: Transforms for image compression

Transforms used in image compression

- Most of transforms are **signal independent** except for **KLT**
- **Karhunen-Loéve transform** (KLT)
  - Transform **minimizes** geometric mean of **variance** of transform **coefficients** – **maximizes transform coding gain**
    - **Transform** matrix can be determined using **autocorrelation matrix** of **random process**
    - Transform **matrix** is **signal dependent** and it **has to be sent** to **receiver**
- **Discrete cosine transform** (DCT)
  - Transformation **matrix** formed with **cosines**

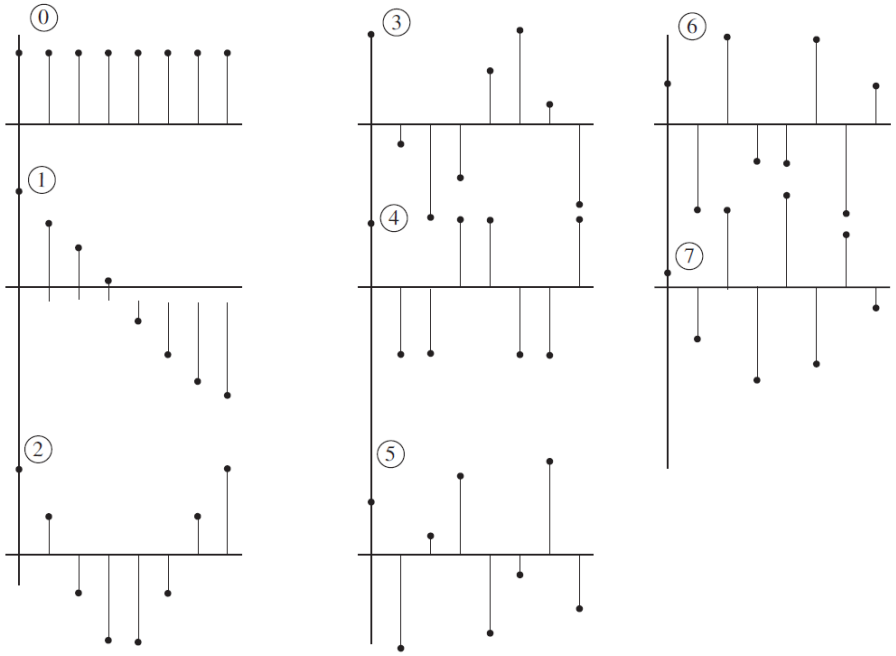
$$[\mathbf{C}]_{i,j} = \begin{cases} \sqrt{\frac{1}{N}}\cos\frac{(2j+1)i\pi}{2N} & i = 0, j = 0,1,\dots,N-1 \\ \sqrt{\frac{2}{N}}\cos\frac{(2j+1)i\pi}{2N} & i = 1,\dots,N-1, j = 0,1,\dots,N-1 \end{cases}$$

    - DCT is **similar** to **DFT** but **achieves better compression performance**

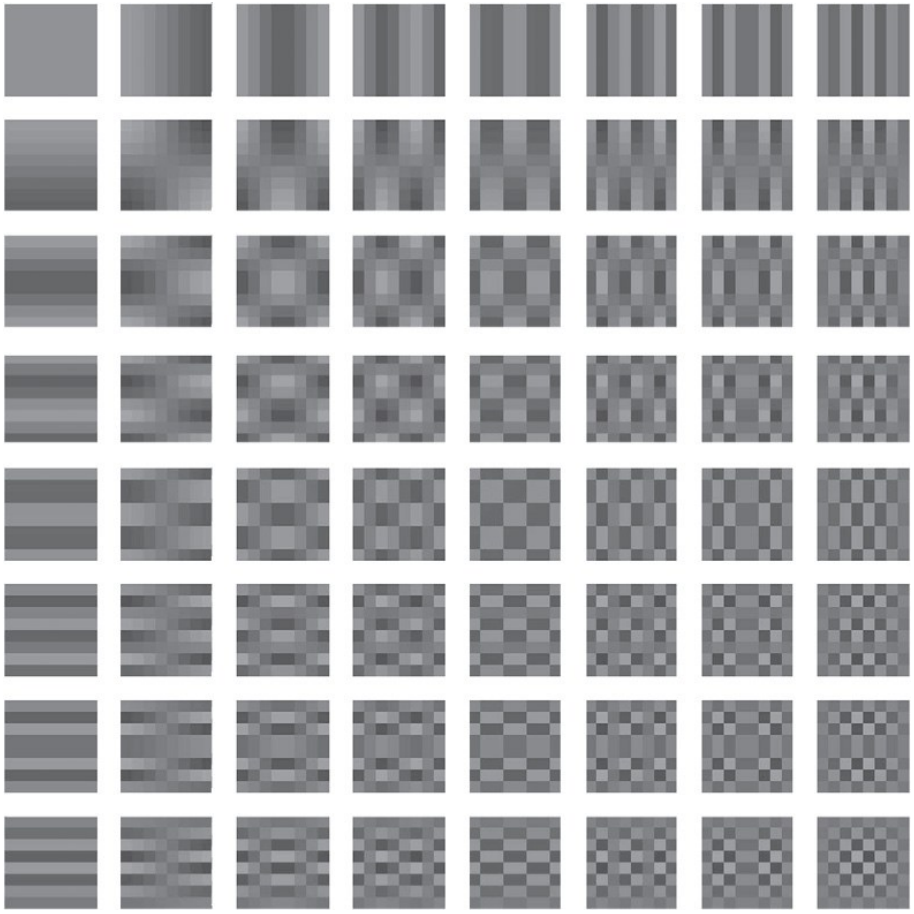
Discrete cosine transform (DCT)

- First **column** of **transformation matrix** and **basis matrix** for **DCT**

First column of transformation matrix
with respective row numbers



Basis matrices for DCT



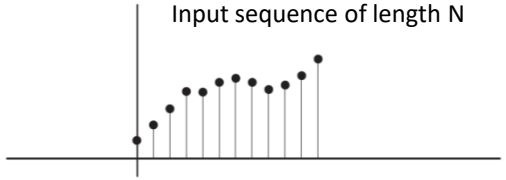
$$[\mathbf{C}]_{i,j} = \begin{cases} \sqrt{\frac{1}{N}}\cos\frac{(2j+1)i\pi}{2N} & i = 0, j = 0,1,\dots,N-1 \\ \sqrt{\frac{2}{N}}\cos\frac{(2j+1)i\pi}{2N} & i = 1,\dots,N-1, j = 0,1,\dots,N-1 \end{cases}$$

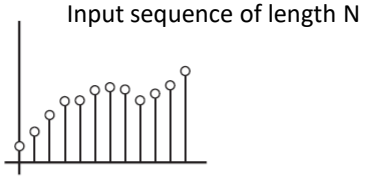
© Multimedia Technology Group

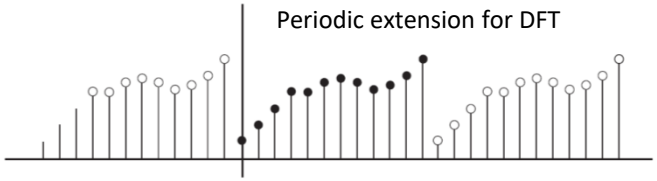
Transform coding: Transforms for image compression

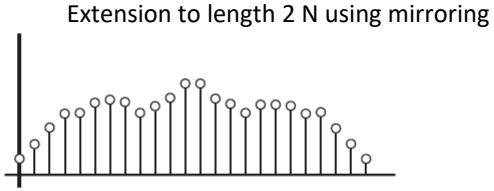
Discrete cosine transform (DCT)

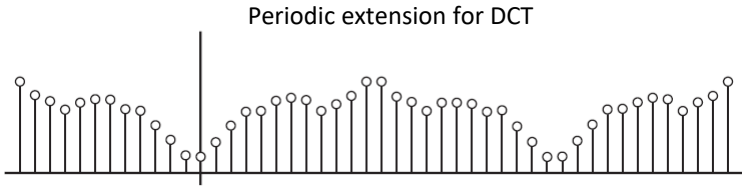
- **DCT** has **more convenient properties** than **DFT**











  - DFT requires **nonzero coefficients** of **high frequency components** for description of **discontinuities**
  - **Discarding** these **coefficients** in compression results in **distortion**
  - **DCT** can be obtained from **DFT** using **original sequence mirroring** from N to 2 N
  - **DFT application** assumes **periodic extension** for **DCT** calculation
  - Periodic extension **does not cause discontinuities**
  - **DCT decorrelation performance** close to **KLT** for **typical image signals**

Transform coding: Transforms for image compression

Discrete sine transform (DST)

- **DST** is **complementary** transform to **DCT** and is suitable for **decorrelation** of **signals with weaker correlation**
  - Used in **image** and **audio compression**
  - DST transformation matrix

$$[\mathbf{S}]_{i,j} = \sqrt{\frac{2}{N+1}} \sin \frac{\pi(i+1)(j+1)}{N+1}, \qquad i,j = 0,1,\dots,N-1$$

Discrete Walsh-Hadamard transform (DWHT)

- Very **simple implementation** and **less efficient than DCT**
  - Using **Hadamard matrix** $\mathbf{H}$ of order N and **dimensions** $N \times N$, for which applies $\mathbf{H}\mathbf{H}^T = N\mathbf{I}$, where $\mathbf{I}$ is **identity matrix** of dimensions $N \times N$
  - **Hadamard matrices** with dimension in **powers of two** can be constructed

$$\mathbf{H}_{2N} = \begin{bmatrix} \mathbf{H}_N & \mathbf{H}_N \\ \mathbf{H}_N & -\mathbf{H}_N \end{bmatrix}, \text{ where } \mathbf{H}_1 = [1]$$

  - **Transform matrix** obtained **sorting rows** according to **number** of **sign changes**
  - **Normalization** performed to obtain $\mathbf{H}\mathbf{H}^T = \mathbf{I}$
  - Matrix **contains only** ± 1, **implementation** with **addition/subtraction operators**

Transform coding: Quantization and coding of coefficients

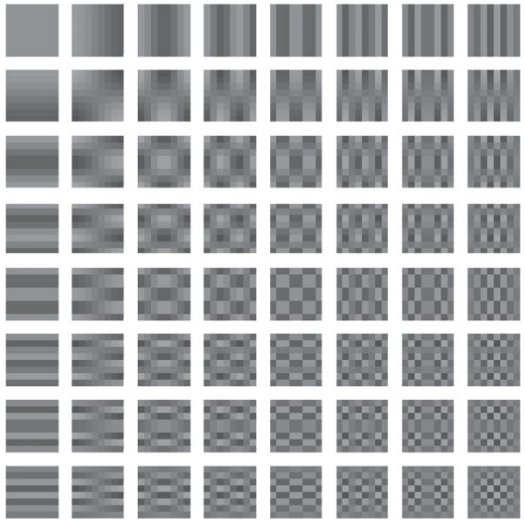
Quantization and coding of transform coefficients

- **Amount** of **information** carried by the coefficients **varies**
  - **Assigning different number** of **bits** to different **coefficients**
- Procedure of **bit allocation** based on **zonal mask**
  - Estimation of **transform coefficients variances**
  - **Variance** describes **amount** of **information** carried by the coefficients
  - Larger variance → more bits, smaller variance → less bits
  - **Disadvantage** of **content dependence** of **bit allocation**

Example of bit allocation map for 8x8 DCT in image compression

8	7	5	3	1	1	0	0
7	5	3	2	1	0	0	0
4	3	2	1	1	0	0	0
3	3	2	1	1	0	0	0
2	1	1	1	0	0	0	0
1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

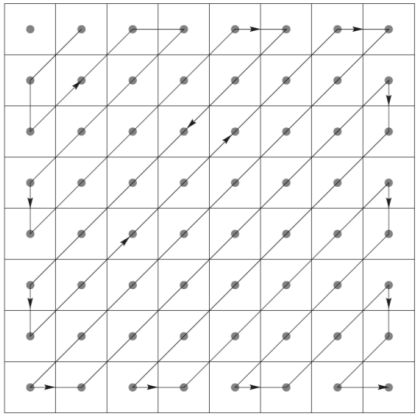
Basis matrices for DCT



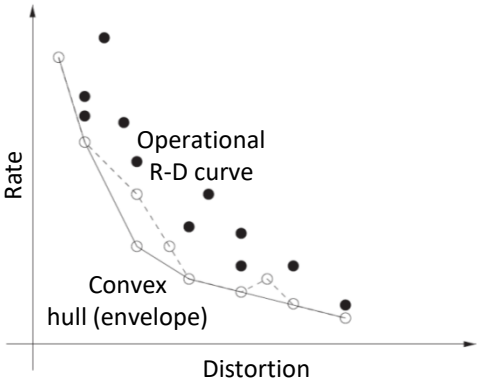
Transform coding: Quantization and coding of coefficients

- **Bit allocation** based on **threshold coding**
  - **Not decided** in **advance** which **coefficients** will be **preserved**
  - **Coefficients** below threshold **discarded**, others **quantized** and **transmitted**
  - For **image** compression and **2D DCT zig-zag** scanning
  - With described **readout** of **quantized coefficients** is large number of **zero** coefficients
  - Coefficients of **high frequency components** reach **lower level**
  - Last **zero coefficients** are **discarded** and **end-of-block** (**EOB**) symbol is **transmitted**

Zig-zag scanning



- **Bit allocation based on functional analysis**
  - **Parameters optimization** using **rate-distortion** (**R-D**) analysis
  - **Quantizer** is determined for **each bit rate** to **minimize error**
  - Determination of **optimal bit allocation using brute force**

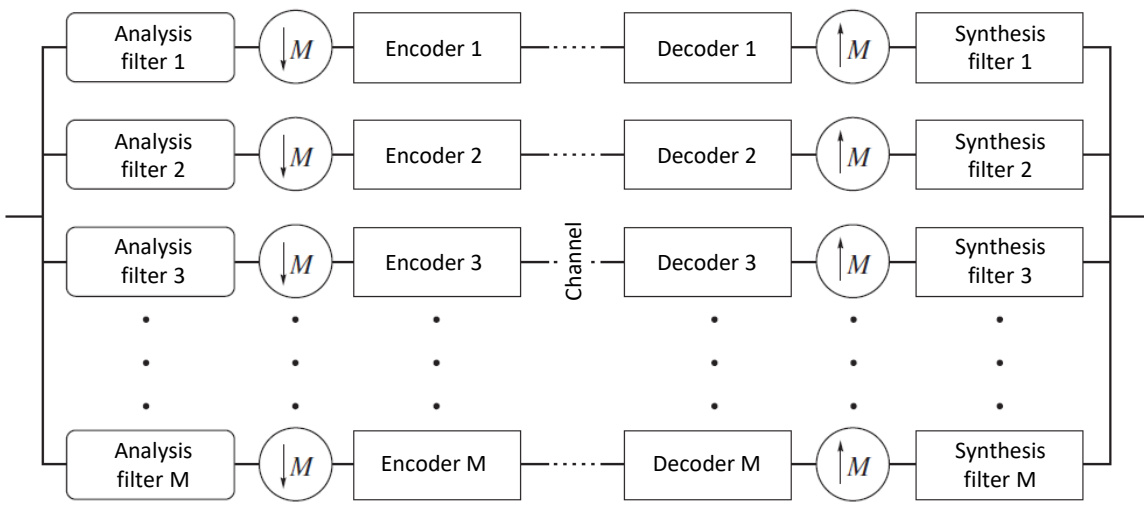


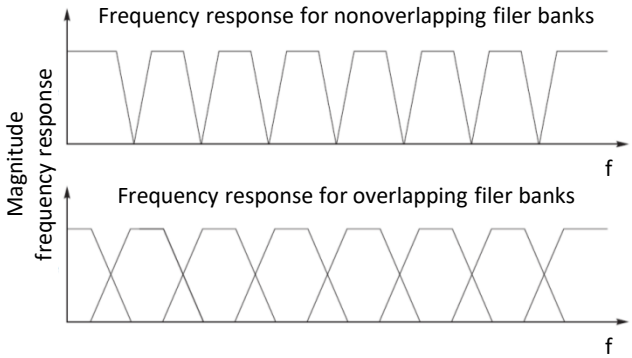
Subband coding: Introduction

Subband coding

- **Decomposition** of **signal** into **more bands** using **filtration**
  - **Compression scheme** (e.g. scalar and vector quantization, differential encoding, transform coding) is **optimized** for **signal characteristics**
- **Advantages** of subband coding
  - **No division** into **blocks** (as for DCT) – **artifacts suppression**
  - **Encoding** optimized for each **band** considering its **characteristics**
  - Properties of **receiver** (observer) can be taken into account for **various bands**

Block diagram for subband coding system





Subband coding: Analysis, quantization and coding, synthesis

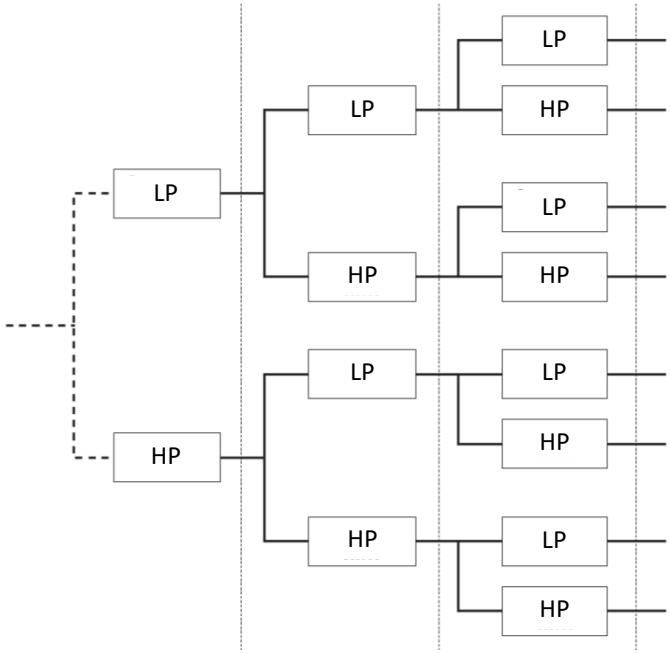
- **Subband coding consists of three main processes**
  - **Analysis** – **filter bank** decomposition covering **source frequency range**
    - **Filter frequency responses** can be **overlapping** or **nonoverlapping**
    - **Filter outputs** from **filters** are **decimated/downsampled**
    - **Downsampling** can be performed thanks to fulfilment of **Nyquist condition**
  - **Quantization** and **coding** – **compression scheme** and **bit allocation**
    - **Respective bands** carry **different amount** of **information** about original signal
    - **Same** or **different number** of **bits** can be **allocated** to **respective bands**
  - **Synthesis** – **signal reconstructed** from **quantized** and **encoded coefficients**
    - **Upsampling** performed to original number of samples
    - **Filtration** performed using **filter bank** and **merging**
  - Crucial impact on **coding properties** is in analysis and synthesis **filter bank**, **bit allocation** and **coding scheme**
- Advantage in exploiting **frequency dependent visual system**
  - **Artefacts** can appear in **bands** with **lower sensitivity**

Subband coding: Analysis, quantization and coding, synthesis

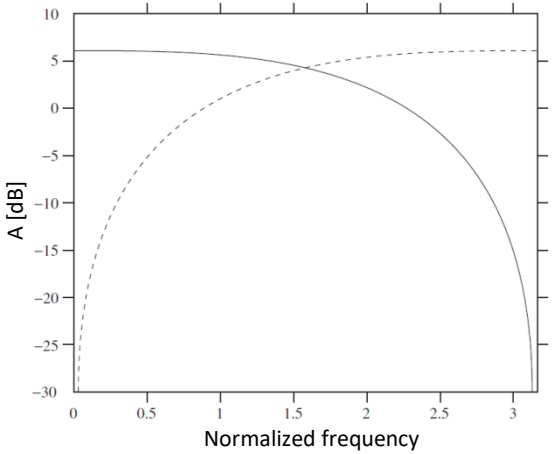
Decomposition using multiresolution filter bank

- Common **usage** of **recursive band division** using LP and HP filters
  - Quadrature **mirror filters** (QMF)
  - Low-pass LP $H_1(z)$ and high-pass filters HP, $H_2(z) = H_1(-z)$

Multiple band quadrature mirror filter bank



Magnitude characteristics of mirror filters



- **Decomposition** can be **based** on **decimation order**
  - **Underdecimated**, **critically decimated** (most common) or **overdecimated**

Subband coding: Application in image coding

Application in image coding

- Using **separable 2D filters**
  - **Rows** and **columns** of image matrix **filtered separately** using **LP** and **HP** filters
  - **Decimation** of **order 2** used at **output**
  - **Two images** $\frac{N}{2} \times N$ obtained after **row filtration** of images with size $N \times N$
  - Each **image** $\frac{N}{2} \times N$ filtered in **columns** obtaining **two images** $\frac{N}{2} \times \frac{N}{2}$
- **Decomposing** image
  - **LP** and **HP filtration** of **rows** and **decimation** with **order 2**, **2 images** obtained
  - **LP** and **HP filtration** of **columns** and **decimation** with **order 2**, **4 images** obtained
- Resulting **4 images** denoted
  - **LL** (*low-low*) – approximation
  - **LH** (*low-high*) – horizontal details
  - **HL** (*high-low*) – vertical details
  - **HH** (*high-high*) – diagonal details

Image decomposition into bands with notation

LL Approximation	**HL** Vertical details
LH Horizontal details	**HH** Diagonal details

Subband coding: Application in image coding

Subband coding

- **Subbands encoded** usually using **scalar quantization**, **vector quantization** or **differential encoding**

Image decomposition using different filter types

Johnston filter 8

Johnston filter 16

Smith-Barnwell filter 8

- Example of average **allocation** to **1 bit per pixel** for two bands (**LL**, **HL**) with **DPCM application** in **LL band** and **scalar quantization** in **HL band**

0.5 bits per pixel
Johnston filter 8

0.5 bits per pixel
Smith-Barnwell filter 8

Wavelets: Introduction

Wavelet transform

- **Wavelet** can be used for **signal decomposition**
  - **Wavelet based compression system** similar to **subband coding**
  - Output signal **decomposed** using **wavelet filter bank**
  - Decomposition result then **subsampled**, **quantized** and **encoded**
  - **Reconstruction** based on **interpolation** and **synthesis** using filter bank
- **Comparison to Fourier transform**
  - Fourier transform with **no localization** in **time**

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-j\omega t} dt$$

  - **Spectrum variances** in **time** for **nonstationary signals**
    - This information provided by **short-term FT** (STFT)
    - **Sliding window** application $g(\tau)$ **– issue** of **constant widow length**

$$F(\omega, \tau) = \int_{-\infty}^{\infty} f(t) g(t-\tau) e^{-j\omega t} dt$$

Wavelets: Introduction

- **Continuous wavelet transform** (CWT - Continuous WT)
  - Can be **written** as

$$w_{a,b} = \int_{-\infty}^{\infty} \psi_{a,b}(t) f(t) dt$$

$\psi_{a,b}(t)$ is **family** of **functions generated** from **mother function** $\psi(t)$

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}} \psi\left(\frac{t-b}{a}\right)$$

where a is **scaling** parameter, b **shift** parameter.

  - For **discrete version** selecting

$$a = a_0^{-m}, \qquad b = n b_0 a_0^{-m}$$

where m and n are **integers**, **usually** $a_0 = 2$ and $a_0 = 1$.

  - For **wavelet set** applies

$$\psi_{m,n}(t) = 2^{\frac{m}{2}} \psi(2^m t - n), \qquad m, n \in \mathbb{Z}$$

and **wavelet transform** in **discrete time** DTWT (Discrete Time WT)

$$w_{m,n} = 2^{\frac{m}{2}} \int_{-\infty}^{\infty} \psi_{a,b}(t) \psi(2^m t - n) dt \text{ and } f(t) = \sum_m \sum_n w_{m,n} \psi_{m,n}(t)$$

© Multimedia Technology Group

Wavelets: Introduction

Scaling function

- **Scaling** function $\phi(t)$ related to **mother function** and using linear combination

$$f(t) = \sum_k a_k \, \phi(t-k)$$

where **Haar function** is simplest scaling function

$$\phi(t) = 1 \text{ for } 0 \leq t < 1 \text{ and } 0 \text{ otherwise.}$$

- **Scaling functions** can be similarly obtained form mother function

$$\phi_{j,k}(t) = 2^{\frac{j}{2}} \phi\left(2^j t - k\right)$$

where first index j determines resolution and second index k shift.

- **Multiresolution analysis** (MRA)

$$\phi(t) = \sum_k h_k \sqrt{2} \, \phi(2t-k)$$

can be **implemented** using **filters**.

Wavelets: Decomposition using filtration

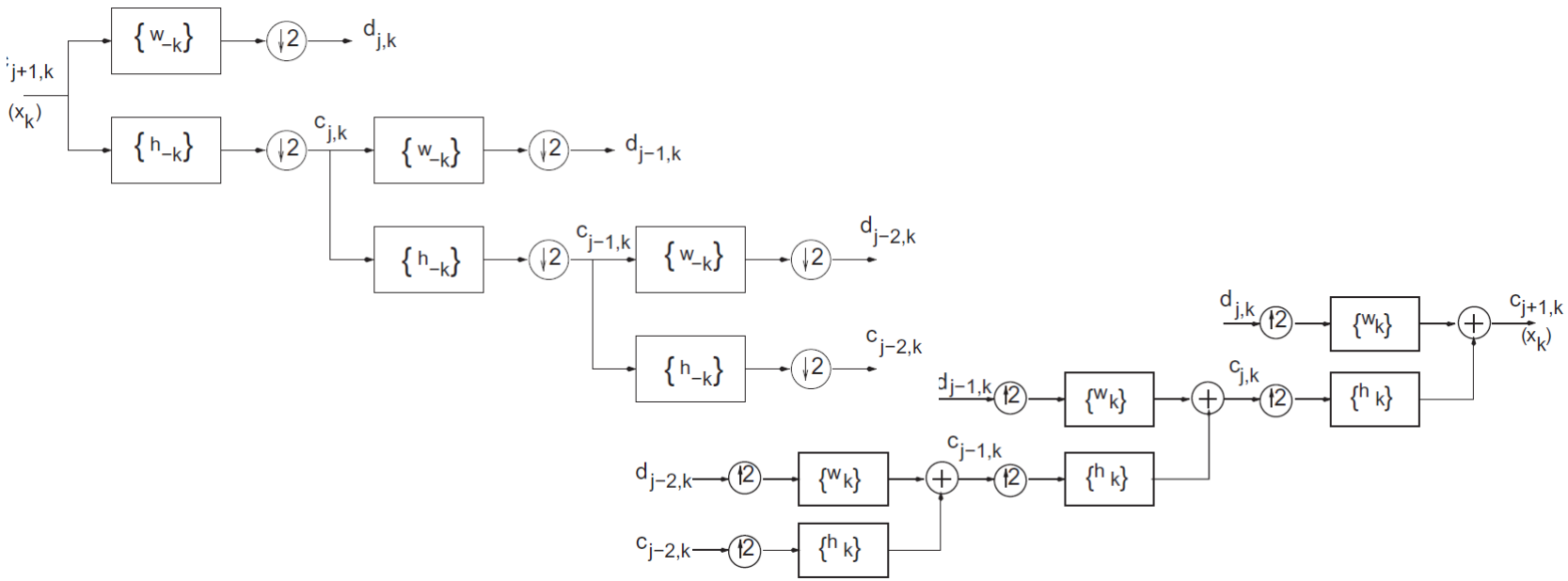
❖ Decomposition implemented using filtration

- **Multiresolution analysis** (MRA)

$$\phi(t) = \sum_k h_k \sqrt{2}\, \phi(2t - k)$$

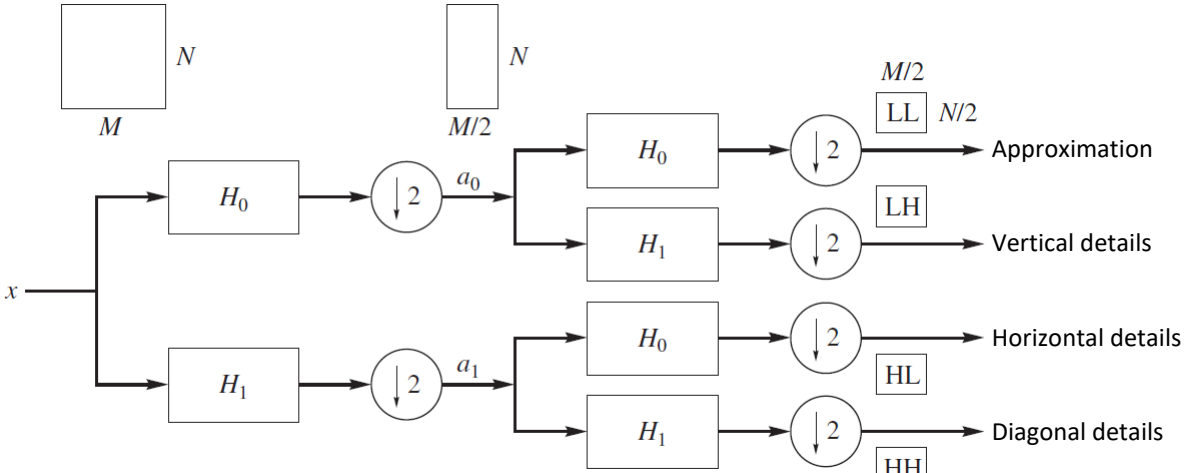
can be **implemented** using **filters**.

- Filters with **impulse response** $\{h_k\}$ and $\{w_k\}$ and **example** of **wavelet decomposition** and **reconstruction** to third level



Wavelet transform based image compression

- Various **compression schemes** using **wavelets**
  - Standard **JPEG 2000** utilizes **wavelets** for **instead** of **DCT** in **JPEG**
- **Signal decomposition using separable filters**
  - **Filtration** and **decimation** performed at first in **rows**
  - Followed by **filtration** and **decimation** in **columns**
  - **Decomposition** can be **extended** to **further levels**



Decomposition of NxM image

First-level decomposition

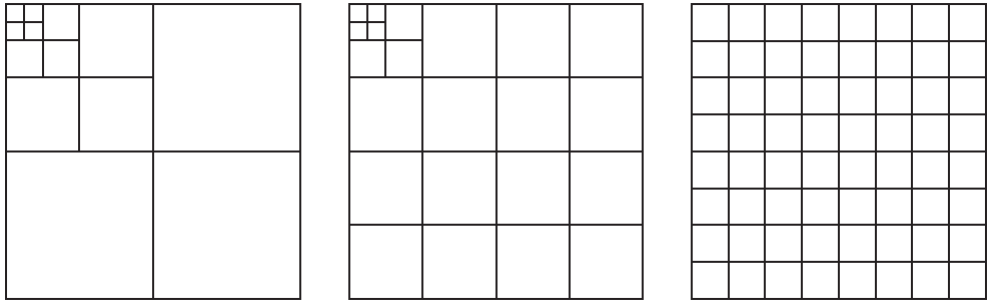
	M/2	M/2
N/2	LL	LH
N/2	HL	HH

© Multimedia Technology Group

Wavelets: Application to image compression

Various decomposition structures

- Three popular **subband structures**

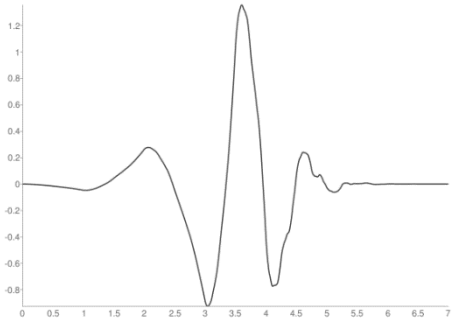


Most common structure

- **Decomposition applied on image**
  - **Various strategies** for **coefficients quantization**

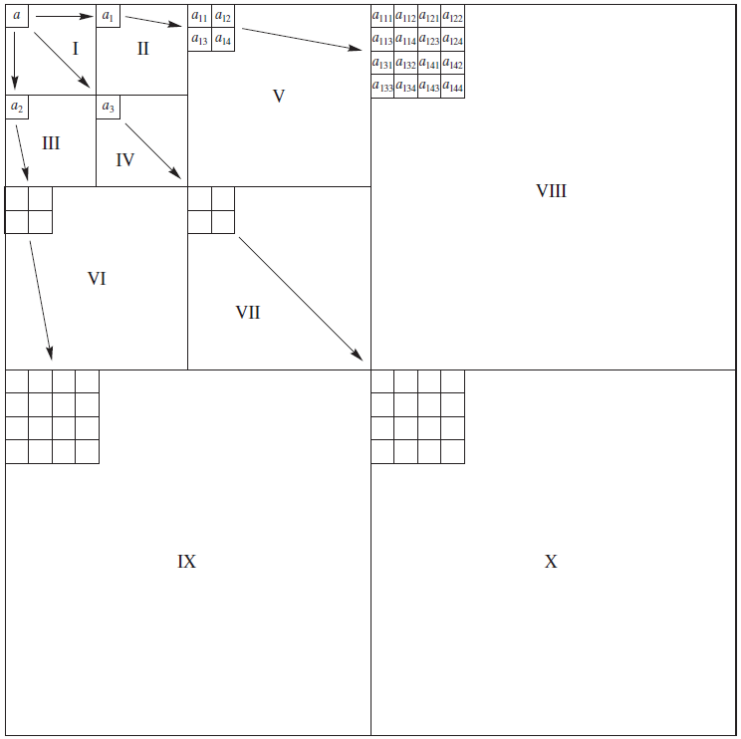
Decomposition using Daubechies 4 wavelet

Daubechies 4 wavelet



Embedded Zerotree Wavelet (EZW) coder

- **Quantization** and **coding** scheme with **wavelet decomposition**
  - Utilizes **spatial coefficients collocation** in respective bands
  - Coefficient in **lower subband** related to **more coefficients** in higher
- Example **decomposition** to **10 bands**
  - Coefficient a in Band I represents same spatial location as a_1 in Band II, coefficient a_2 in Band III, coefficient a_3 in Band IV
  - Coefficient a_1 represents same spatial location as a_{11}, a_{12}, a_{13} and a_{14} in Band V
  - Each of these pixels represents same location as 4 pixels in band VIII

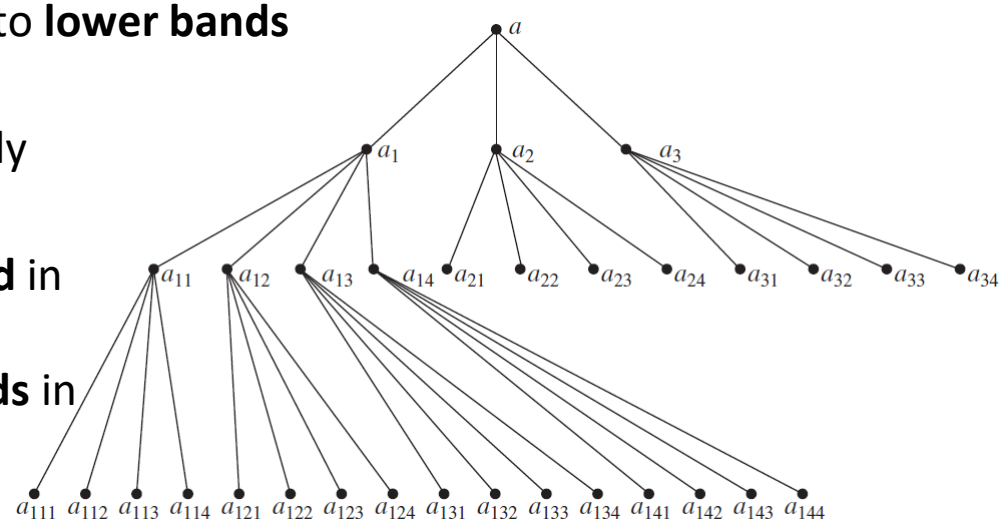


© Multimedia Technology Group

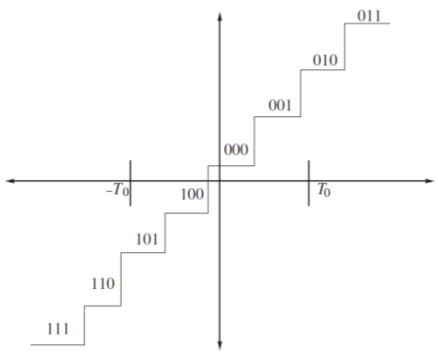
Wavelets: Embedded Zerotree Wavelet

Dependencies described using tree structure

- **Coefficient** a forms **root** for **64 coefficients** in various bands
  - **Most** of **energy** compacted to **lower bands** close to **tree root**
  - **Coefficient amplitude** usually **increases** closer to **root**
  - **Coefficients under threshold** in **lower bands** correspond to **coefficients under thresholds** in **higher** bands



- Dependency **utilized** in **quantization**
  - If all **coefficients** from given root have **magnitude lower than** T_0, then **all coefficients** in tree can be quantized to **2 bits per sample**
  - **Signalized** to **decoder**

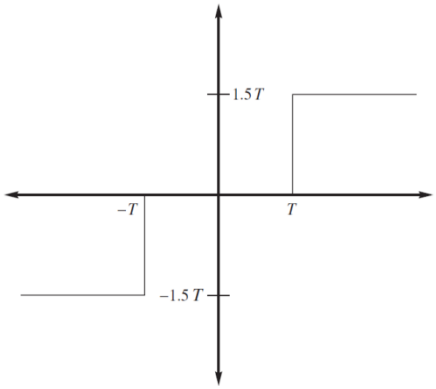


Example of 3-bit midrise quantizer

Wavelets: Embedded Zerotree Wavelet

EZW algorithm

- **Coefficient** with magnitude **lower than threshold** is **insignificant**
  - If also all its **descendants** have **magnitude lower than threshold** then it is called **zerotree root**
  - **Isolated insignificant coefficient** can appear (**isolated zero**)
- **Multiple-pass** algorithm with **two steps** in each pass
  - **Dominant** pass – **significance map encoding**
  - **Refinment** pass
- **Threshold calculated** $T_0 = 2^{\lfloor \log_2 c_{max} \rfloor}$ for largest coefficient c_{max}
  - With next pass **threshold** set to **half** $T_i = \frac{1}{2} T_{i-1}$
  - With each pass **coefficient labeled** as
    - **significant positive – sp**
    - **significant negative – sn**
    - **zerotree root – zr**
    - **isolated zero – iz**
  - **Significant** coefficients fall in **outer levels** of quantizer and are assigned $\pm 1.5 T_i$

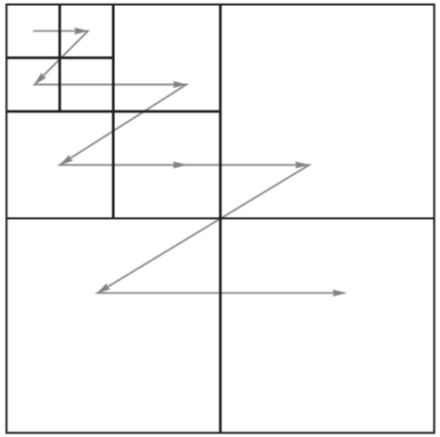


Significance map encoding using quantizer

Wavelets: Embedded Zerotree Wavelet

❖ EZW algorithm

- **Significant coefficients** are **included** in **list**
  - **Coefficients** are **refined** in **refinement pass**
  - Determination if coefficient lies in **upper** or **lower interval half** $[T, 2T]$
  - **Threshold** T value is **lowered** and **interval narrowed**
  - **Difference** between **coefficient value** and its **reconstruction calculated**
  - **Difference** is **quantized** with reconstruction value $\pm T/4$
  - **Quantized value** added to **actual reconstruction** as **correction**
- Coefficients **scanned** according to diagram



Scanning of wavelet coefficients for EZW algorithm encoding

EZW algorithm – example

- **Decomposition to 7 bands**

26	6	13	10
− 7	7	6	4
4	− 4	4	− 3
2	− 2	− 2	0

(1) Threshold determination

$$T_0 = 2^{\lfloor \log_2 26 \rfloor} = 16$$

(2) Coefficient 26 larger than 16, sending ***sp***. Next coefficient 6 smaller than 16 and also all its descendants (13, 10, 6, 4). Thus 6 is zerotree root, sending ***zr***. Next coefficient -7 and 7 also ***zr***. Sequence of transmitted labels ***sp zr zr zr***.

(3) Each label requires 2 bits, 8 bits used. Only coefficient for refinement scan is $L_s = \{26\}$. Reconstructed value is $1{,}5T_0 = 24$ and for reconstructed bands apply.

24	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

(4) Refinement scan for significant coefficients. Difference $26 - 24 = 2$, quantization with reconstruction level $\pm T_0/4$, thus $24 + 4 = 28$. Transmitting correction requires 1 bit, overall 9 bits and achieved reconstruction.

28	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

(5) Lowering threshold to half $T_1 = 8$. Coefficient scanning labeled as insignificant. First coefficient lower than threshold, descendants (13, 10) no,

*	6	13	10
− 7	7	6	4
4	− 4	4	− 3
2	− 2	− 2	0

thus isolated zero ***iz***. Further (-7, 7), (13, 10) and (6, 4), thus **iz zr zr sp sp iz iz**.

© Multimedia Technology Group

Wavelets: Embedded Zerotree Wavelet

❖ EZW algorithm – example

(5) Each label requires 2 bits, 14 bits used and overall 23 bits. Reconstructed value is $1{,}5T_1 = 12$ and for reconstructed bands apply. Adding significant coefficients for subordinate pass $L_s = \{26, 13, 10\}$.

28	0	12	12
0	0	0	0
0	0	0	0
0	0	0	0

(6) Subordinate pass for significant coefficients. Difference $26 - 28 = -2$, $13 - 12 = 1$ and $10 - 12 = -2$ quantization with reconstruction level $\pm T_1/2 = \pm 2$, thus -2, 2 and -2. After correction.

26	0	14	10
0	0	0	0
0	0	0	0
0	0	0	0

(7) In one more pass obtaining coefficients not labeled as significant. Sequence of transmitted labels is ***sp sn sp sp sp sp sn iz iz sp iz iz iz***.

⋆	6	⋆	⋆
− 7	7	6	4
4	− 4	4	− 3
2	− 2	− 2	0

(8) Pass cost is 26 bits. Reconstruction and added significance coefficients are $L_s = \{26, 13, 10, 6, -7, 7, 6, 4, 4, -4, 4\}$.

(9) Encoding continues in same manner until bit budget exhausted or reaching some other criterion.

26	6	14	10
−6	6	6	6
6	−6	6	0
0	0	0	0

- **Optimality** after interrupted encoding
  - Quality **increases** with **number** of **transmitted bits**
  - Usually **arithmetic encoder** used for further **performance improvement**

© Multimedia Technology Group

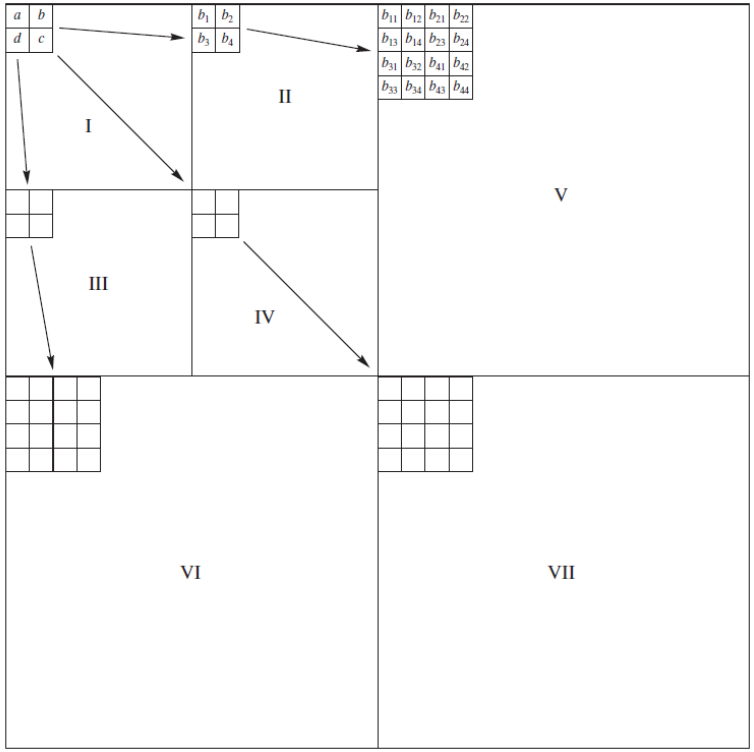
Set Partitioning in Hierarchical Trees (SPIHT) coder

Generalization of EZW algorithm

- **SPIHT** uses **partitioning** of **trees** with **insignificant coefficients** in **larger subsets**
  - **Decisions** on partitioning in binary form **transmitted** to **decoder**
- Coefficients in **lower subband** related to **more coefficient** in **higher**

SPIHT structure description

- Coefficients **grouped into** 2×2 arrays
- Each **array** (except for Band I) is **descendent** of coefficients in **lower band**
- **All coefficients** except for in **top-left** corner form **root nodes**



© Multimedia Technology Group

JPEG standard: Transform and quantization

JPEG standard

Most common standard for image compression

- Among **most successful standards** ever
  - Cooperation between **International Standards Organization** (ISO) and **ITU-T**
  - **DCT** based **transform coding**

Transform

- Subtracting **half** of **dynamic range** 2^{P-1} from **each pixel** (128 for 8bpp)
- **Values** before transform in **range** between – 128 and 127
- Division of **image** into **blocks** 8×8 and **application** of forward **2D DCT**
  - If **image dimensions** are not multiples of 8 then **extension repeating rows** and **columns**
  - These **pixels removed** after **reconstruction**

Quantization

- Uniform **quantization** of coefficients using **quantization matrix**

$$l_{i,j} = \left\lfloor \frac{\theta_{i,j}}{Q_{i,j}} + 0.5 \right\rfloor$$

$\theta_{i,j}$ is element of quantization matrix at (i, j) and $\lfloor x \rfloor$ is largest integer less than x.

DCT and quantization

Block of 8x8 image pixels

52	55	61	66	70	61	64	73
63	59	66	90	109	85	69	72
62	59	68	113	144	104	66	73
63	58	71	122	154	106	70	69
67	61	68	104	126	88	68	70
79	65	60	70	77	63	58	75
85	71	64	59	55	61	65	83
87	79	69	68	65	76	78	94

-2^{P-1}

Suppressed mean value

−76	−73	−67	−62	−58	−67	−64	−55
−65	−69	−62	−38	−19	−43	−59	−56
−66	−69	−60	−15	16	−24	−62	−55
−65	−70	−57	−6	26	−22	−58	−59
−61	−67	−60	−24	−2	−40	−60	−58
−49	−63	−68	−58	−51	−65	−70	−53
−43	−57	−64	−69	−73	−67	−63	−45
−41	−49	−59	−60	−63	−52	−50	−34

2D DCT

Block of 8x8 DCT coefficients

−415	−29	−62	25	55	−20	−1	3
7	−21	−62	9	11	−7	−6	6
−46	8	77	−25	−30	10	7	−5
−50	13	35	−15	−9	6	0	3
11	−8	−13	−2	−1	1	−4	1
−10	1	3	−3	−1	0	2	−1
−4	−1	2	−1	2	−3	1	−2
−1	−1	−1	−2	−1	−1	0	−1

Quantization

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Block of 8x8 quantized DCT coefficients

−26	−3	−6	2	2	0	0	0
1	−2	−4	0	0	0	0	0
−3	1	5	−1	−1	0	0	0
−4	1	2	−1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$$l_{i,j} = \left\lfloor \frac{\theta_{i,j}}{Q_{i,j}} + 0.5 \right\rfloor$$

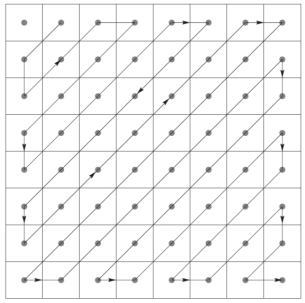
© Multimedia Technology Group

JPEG standard: Transform, quantization and encoding

Quantization

- Quantization **step increases from DC to AC**
  - Quantization **error larger** for **higher** spatial **frequencies**
  - **Coefficients** determined **based** on **observer characteristics**
  - **Larger quantization step** for **less significant coefficients**
- **Zig-zag** scanning – **long zero runs**
  - After **last nonzero** coefficient **EOB** (**End of Block**) code
- **Quantization matrix** modification – **quality** and **bitrate**

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99



Encoding

- **DC coefficients** are **very similar** after **DCT**
  - **Decorrelation** using **differential encoding**
- **Difference encoding** using **table**
  - First **category determined** then **code assigned**
  - **Category** encoded using **Huffman** encoder and **value in range**

0			0			
1			−1	1		
2		−3	−2	2	3	
3	−7	···	−4	4	···	7
4	−15	···	−8	8	···	15
5	−31	···	−16	16	···	31
6	−63	···	−32	32	···	63
7	−127	···	−64	64	···	127
8	−255	···	−128	128	···	255
9	−511	···	−256	256	···	511
10	−1,023	···	−512	512	···	1,023
11	−2,047	···	−1,024	1,024	···	2,047
12	−4,095	···	−2,048	2,048	···	4,095
13	−8,191	···	−4,096	4,096	···	8,191
14	−16,383	···	−8,192	8,192	···	16,383
15	−32,767	···	−16,384	16,384	···	32,767
16			32,768			

JPEG standard: Encoding

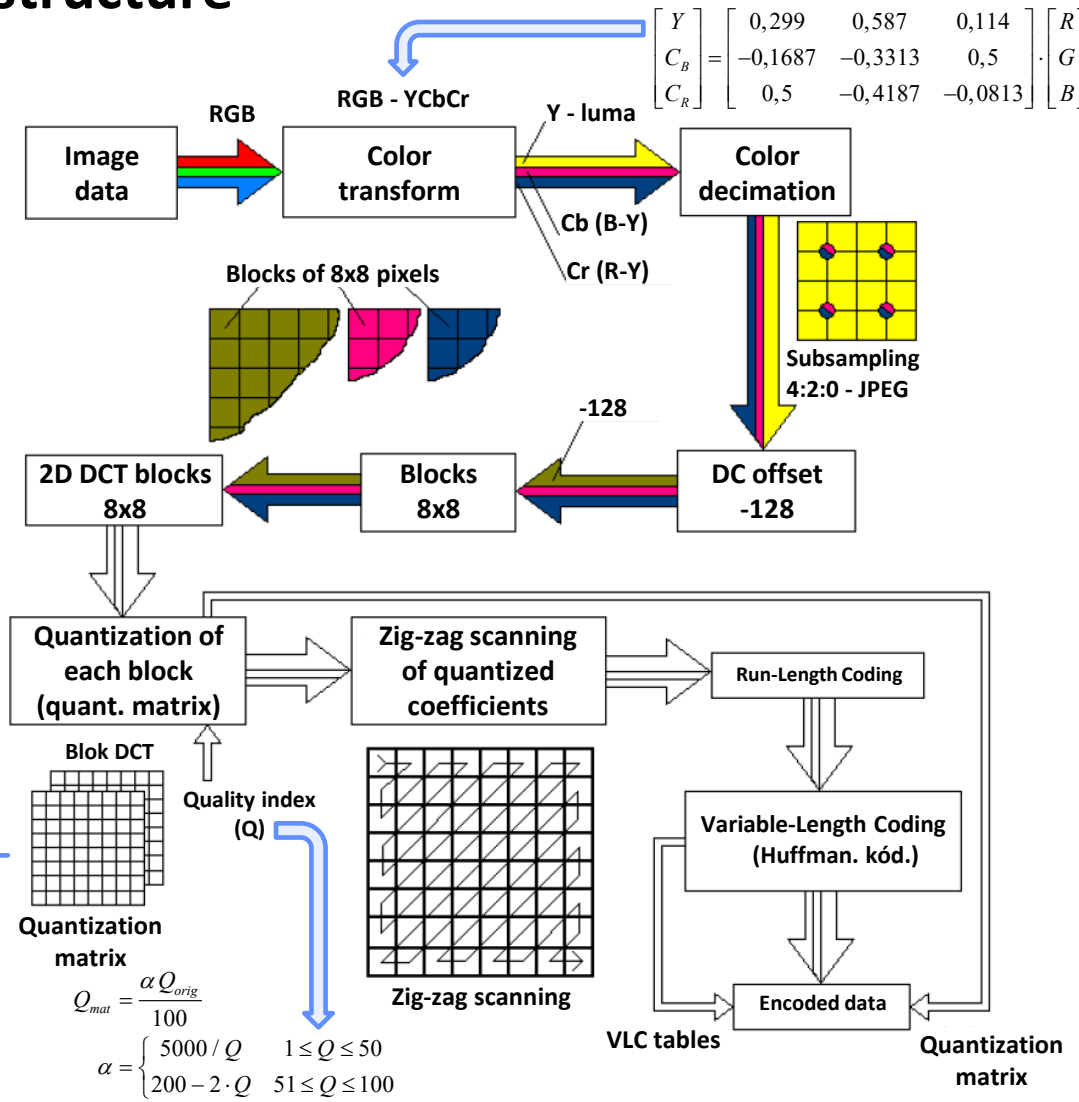
Encoding

- Detail **AC coefficients** encoded **differently than DC**
- **Run length encoding** using **Z/C label**
  - **C** – **category** of **related nonzero coefficient**
  - **Z** – **number of zeros** after previous **nonzero coefficient**
- **Huffman code** assigned to **Z/C** according to **table**
  - For example **AC coefficient** from **F category** (Hexa) with **15 zeros** before
    - Assigned **label F/3** and **codeword 1111111111110111** followed by **3 bits for value**
- Two special **Z/C labels**
  - Denoted **EOB** for **last nonzero coefficient**
  - **ZRL** number of zeros **larger than 15**

Z/C	Codeword	Z/C	Codeword	$\cdots$	Z/C	Codeword
0/0 (EOB)	1010			$\cdots$	F/0 (ZRL)	11111111001
0/1	00	1/1	1100	$\cdots$	F/1	1111111111110101
0/2	01	1/2	11011	$\cdots$	F/2	1111111111110110
0/3	100	1/3	1111001	$\cdots$	F/3	1111111111110111
0/4	1011	1/4	111110110	$\cdots$	F/4	1111111111111000
0/5	11010	1/5	11111110110	$\cdots$	F/5	1111111111111001
$\vdots$	$\vdots$	$\vdots$	$\vdots$		$\vdots$	

JPEG encoder structure

$$\begin{bmatrix} Y \\ C_B \\ C_R \end{bmatrix} = \begin{bmatrix} 0,299 & 0,587 & 0,114 \\ -0,1687 & -0,3313 & 0,5 \\ 0,5 & -0,4187 & -0,0813 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$



$$\begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

$$\begin{bmatrix} 17 & 18 & 24 & 47 & 99 & 99 & 99 & 99 \\ 18 & 21 & 26 & 66 & 99 & 99 & 99 & 99 \\ 24 & 26 & 56 & 99 & 99 & 99 & 99 & 99 \\ 47 & 66 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \end{bmatrix}$$

$$Q_{mat} = \frac{\alpha Q_{orig}}{100}$$

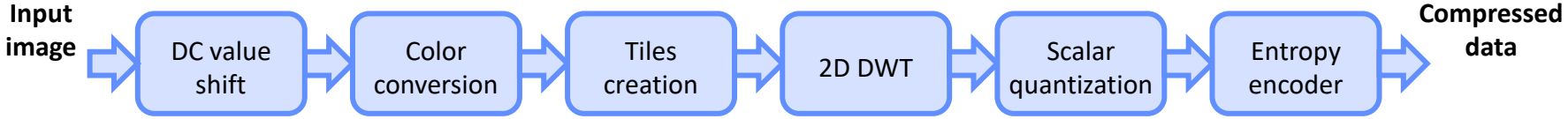
$$\alpha = \begin{cases} 5000 / Q & 1 \le Q \le 50 \\ 200 - 2 \cdot Q & 51 \le Q \le 100 \end{cases}$$

JPEG 2000 standard: Introduction

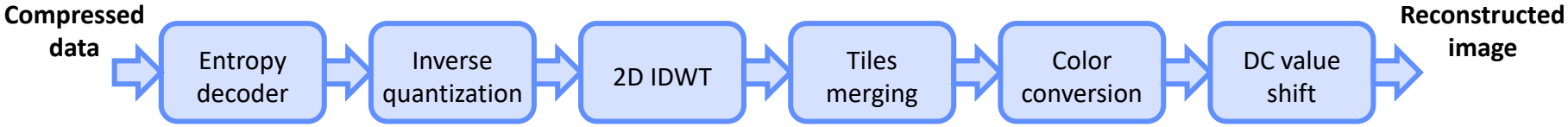
JPEG 2000 standard

- Described in 2000 as **ISO standard** and **ITU-T recommendation**
  - Can be found mostly in **special application** (D-Cinema, medicine, ...)
    - Much **better compression performance** than JPEG (around 0.25 bpp)
    - **Specifies** (1) **decoding** process, (2) bitstream **syntax**, (3) file **format**, (4) provides guidelines for **coder implementation** and (5) proposes **practical implementations**
  - Utilizes **discrete wavelet transform 2D DWT**
  - Extremely **broad standard** – we focus only on **basic parts description**
- **Basic structure of encoder and decoder**

Encoder structure JPEG 2000



Decoder structure JPEG 2000



© Multimedia Technology Group

JPEG 2000 standard: Particular components

DC value shift

- Subtraction of **constant** for **half** of **dynamic range**

$$-2^{P-1}$$

  where P is bit depth of particular component.
- For reconstruction adding **constant** for **half** of **range**

$$+2^{P-1}$$

Conversion of color components

- R, G, B components **highly correlated** – **inefficient independent coding**
- Conversion to **suitable color space** – decorrelation and **performance gain**
- Variant for **lossy compression**

$$\begin{bmatrix} Y \\ C_b \\ C_r \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.144 \\ -0.16875 & -0.33126 & 0.5 \\ 0.5 & -0.41869 & -0.08131 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

- Variant for **lossless compression**

$$\begin{bmatrix} Y_r \\ V_r \\ U_r \end{bmatrix} = \begin{bmatrix} \left\lfloor \dfrac{R + 2G + B}{4} \right\rfloor \\ R - G \\ B - G \end{bmatrix}$$

JPEG 2000 standard: Particular components

Inverse conversion of color components

- Applied in **compressed image reconstruction**
- Variant for **lossy compression**

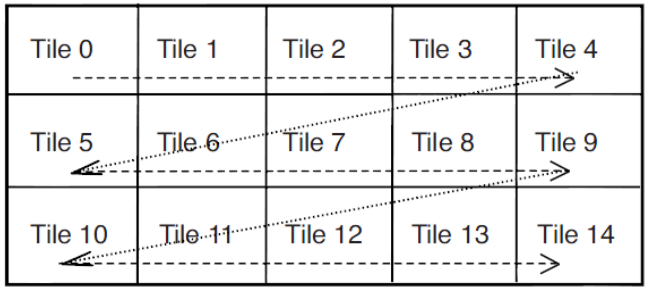
$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1.402 \\ 1 & -0.34413 & -0.71414 \\ 1 & 1.772 & 0 \end{bmatrix} \begin{bmatrix} Y \\ C_b \\ C_r \end{bmatrix}$$

- Variant for **lossless compression**

$$\begin{bmatrix} G \\ R \\ B \end{bmatrix} = \begin{bmatrix} Y_r - \left\lfloor \frac{U_r + V_r}{4} \right\rfloor \\ V_r + G \\ U_r + G \end{bmatrix}$$

Image partitioning into tiles

- **Lower computational complexity**
- Similarly as **JPEG macroblocks**
- **Tiles** – rectangular areas
- Raster **scanning**
- Color components usually **subsampled**



JPEG 2000 standard: Particular components

Transformation of tiles components using DWT

- **Decomposition** performed to level L **using 2D DWT**
  - Using **Daubechies $\frac{9}{7}$ FIR filters** for **lossy compression**
  - Using **Le Gall $\frac{5}{3}$ FIR filters** for **lossless compression**

Scalar quantization of DWT coefficients

- Application of **uniform quantization** - quantization step $Q(b)$ in band b

$$Q(b) = 2^{R_b + g_b - \varepsilon_b}\left(1 + \frac{\mu_b}{2^{11}}\right)$$

  where R_b is **dynamic range** of band b, g_b is **band gain**, ε_b and μ_b are **exponent** and **mantisa** of coefficients.
  - **HL** and **LH** bands in each decomposition levels $g_b = 2$ and **HH** $g_b = 4$
- Performing **quantization** of **coefficients** $Q_b(u, v)$

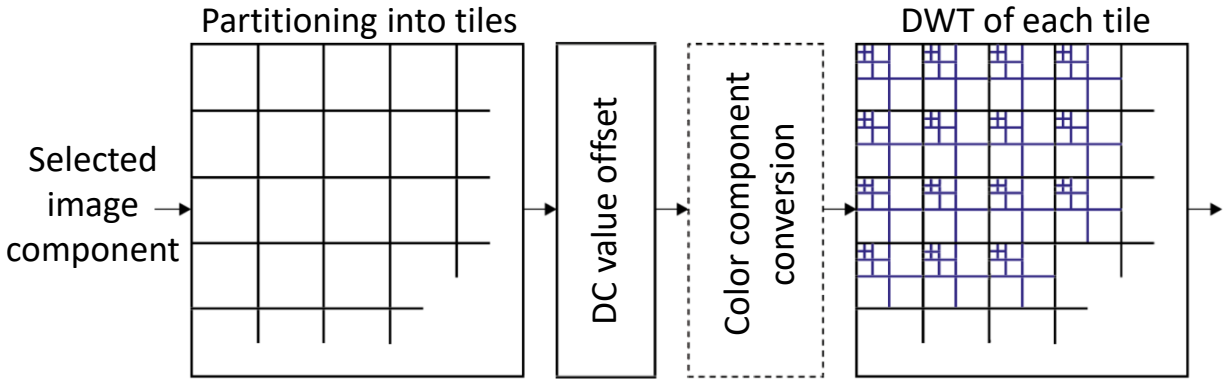
$$X_{qb}(u, v) = \mathrm{sign}(X_b(u, v))\left\lfloor \frac{|X_b(u, v)|}{Q(b)} \right\rfloor$$

- **Quantization can be weighted** depending on **viewing distance**

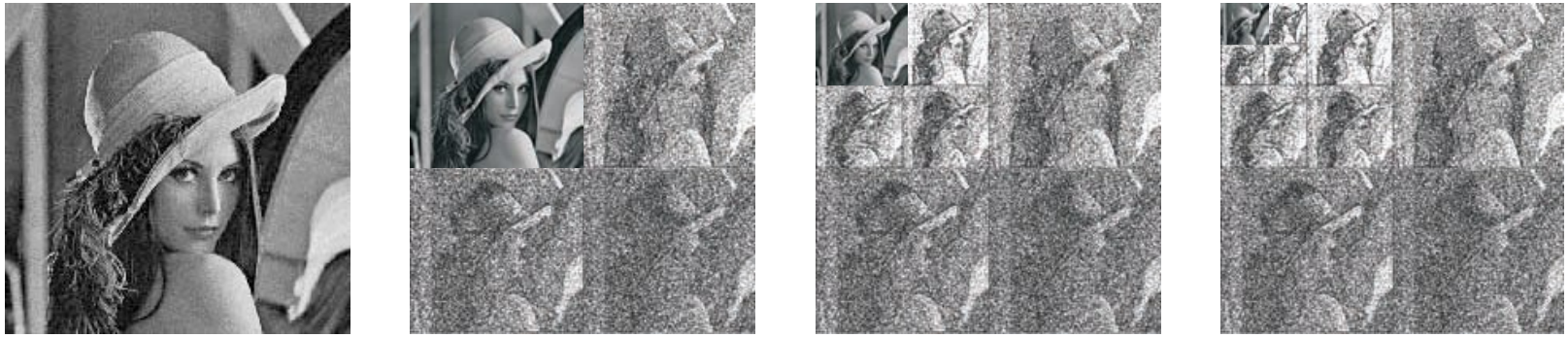
JPEG 2000 standard: Particular components

JPEG 2000 compression system core

- Partitioning into **tiles**, **DC** offset, color component conversion, **2D DWT**



- Example of **dyadic image decomposition** into **third level**



© Multimedia Technology Group

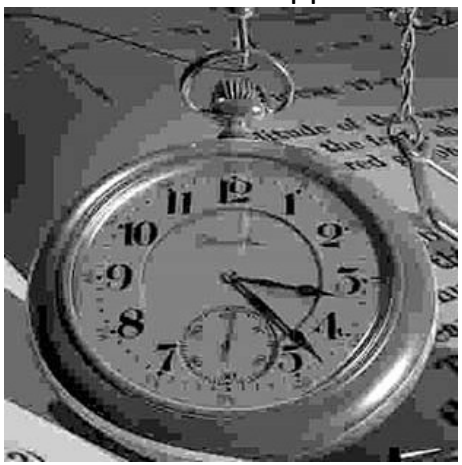
JPEG 2000 standard: Particular components

Comparison of JPEG and JPEG 2000 compression artifacts

- Image compression using **JPEG** and **JPEG 2000** standards
  - JPEG 2000 – blurring artifact, no blockiness as for legacy JPEG

Original

JPEG 0.2 bpp



JPEG 2000 0.2 bpp

JPEG 0.25 bpp

JPEG 2000 0.25 bpp

© Multimedia Technology Group

Recommended literature

- [1] Sayood, K.: **Introduction to Data Compression**, Morgan Kaufmann Publishers, 2006.
- [2] Gonzalez, R. C., Woods, R. E.: **Digital Image Processing**, Pearson, 2007.
- [3] Thyagarajan, K. S.: **Still Image and Video Compression with MATLAB**, Willey, 2011.

Thank you for your attention!

MultiMedia Technology Group

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Radioelectronics

Technická 2
166 27 Prague 6
Czech Republic

© Multimedia Technology Group